영지식증명 톺아보기 5편: Groth16의 마법 - 3개의 점으로 모든 것을 증명하다

Author: 김대익
Reviewed By: —

Disclaimer: 본 글은 Groth16 증명 시스템의 수학적 구조를 다룹니다. 페어링, KZG 커밋먼트, Trusted Setup의 원리를 설명하며, 왜 증명이 단 3개의 타원곡선 점으로 충분한지 이해합니다.

---

1. 들어가며: 왜 Groth16인가

Groth16은 **2016년 EUROCRYPT**에서 Jens Groth가 발표한 zk-SNARK 구성입니다[^1].

논문 제목: **"On the Size of Pairing-Based Non-interactive Arguments"**

이 제목이 핵심을 말해줍니다. Groth16의 목표는 **증명 크기를 최소화**하는 것이었습니다.

Groth16의 성과:

- 증명 크기: **2개 G_1 점 + 1개 G_2 점** (약 128~192 bytes)
- 검증: **3번의 페어링** 연산
- 검증 시간: 회로 크기에 **독립적** (O(1))

2016년 이후, Groth16은 **가장 널리 배포된 zk-SNARK**가 되었습니다[^2]:

- Zcash Sapling (2018~)
- Filecoin PoRep
- Tornado Cash
- Semaphore
- 수많은 zkRollup 프로젝트

왜일까요? 증명 크기와 검증 속도가 **온체인 비용**에 직결되기 때문입니다.

[^1]: Groth, J. (2016). *On the Size of Pairing-Based Non-interactive Arguments*. EUROCRYPT 2016. https://eprint.iacr.org/2016/260 [^2]: Alin Tomescu. *Groth16*. https://alinush.github.io/groth16

---

2. 배경 지식: 쌍선형 페어링

Groth16을 이해하려면 먼저 **페어링(Pairing)**을 알아야 합니다.

2.1 페어링이란?

페어링은 두 타원곡선 그룹의 원소를 입력받아 제3의 그룹 원소를 출력하는 함수입니다[^3]:

```
e: G₁ × G₂ → Gₜ
```

여기서:

- **G_1**, **G_2**: 타원곡선 위의 점들로 이루어진 그룹 (덧셈 연산)
- **G_t**: 유한체의 곱셈 그룹 (곱셈 연산)
- **e**: 페어링 함수

[^3]: Vitalik Buterin. (2017). *Exploring Elliptic Curve Pairings*. https://medium.com/@VitalikButerin/exploring-elliptic-curve-pairings-c73c1864e627

2.2 쌍선형성 (Bilinearity)

페어링의 핵심 성질은 **쌍선형성**입니다:

```
e(aP, bQ) = e(P, Q)^(ab)
```

이 성질이 의미하는 것:

- **지수가 "밖으로 나온다"**: 곡선 위의 스칼라 곱이 G_t에서 지수가 됨
- **교환 가능**: e(aP, bQ) = e(bP, aQ) = e(abP, Q) = e(P, abQ)

2.3 페어링이 가능하게 하는 것

페어링 없이 할 수 없는 것:

- Alice가 aP를 보내고, Bob이 bQ를 보낼 때
- 제3자가 e(aP, bQ) = e(P, Q)^(ab)를 계산할 수 있음
- 하지만 a나 b는 알 수 없음 (이산로그 문제)

zk-SNARK에서의 역할:

- "암호화된 곱셈"을 가능하게 함
- 다항식 관계를 검증 가능하게 함
- **커밋먼트의 일관성 검사**에 필수적

2.4 비대칭 페어링

현대 암호학에서는 **비대칭 페어링** (Type-3)을 사용합니다:

- $G_1 \neq G_2$ (두 그룹이 다름)
- $G_1 \rightarrow G_2$ 또는 $G_2 \rightarrow G_1$로 가는 효율적인 동형사상이 없음
- BLS12-381, BN254가 대표적인 페어링 친화적 커브

---

3. KZG 다항식 커밋먼트

Groth16은 **KZG 커밋먼트**와 유사한 구조를 사용합니다.

3.1 KZG란?

KZG (Kate-Zaverucha-Goldberg, 2010)는 **상수 크기 다항식 커밋먼트**입니다[^4]:

> "다항식 전체가 아닌 단일 점 하나만으로 다항식에 커밋하고, 임의의 점에서의 평가를 증명할 수 있다."

[^4]: Kate, A., Zaverucha, G., & Goldberg, I. (2010). *Polynomial Commitments*. ASIACRYPT 2010. https://www.iacr.org/archive/asiacrypt2010/6477178/6477178.pdf

3.2 Trusted Setup (Powers of Tau)

KZG와 Groth16 모두 **Trusted Setup**이 필요합니다.

비밀 값 τ (tau)를 선택하고, 다음을 공개합니다:

```
G₁ 그룹: [G, τG, τ²G, τ³G, ..., τⁿG]
G₂ 그룹: [H, τH]  (또는 더 많은 거듭제곱)
```

여기서:

- G는 G_1의 생성원
- H는 G_2의 생성원
- **τ는 삭제**됨 (Toxic Waste)

표기법:

- $[a]_1 = aG$ (G_1 그룹의 점)
- $[a]_2 = aH$ (G_2 그룹의 점)

3.3 다항식 커밋

다항식 $f(x) = f_0 + f_1x + f_2x^2 + ... + f_nx^n$에 커밋하려면:

```
C = f(τ)·G = f₀G + f₁(τG) + f₂(τ²G) + ... + fₙ(τⁿG)
```

핵심 통찰:

- 우리는 τ를 모름
- 하지만 τ^iG 값들은 알고 있음 (공개 파라미터)
- 따라서 f(τ)·G는 계산 가능!

이것이 **"암호화된 평가"**입니다. f(τ)의 실제 값은 모르지만, f(τ)·G는 계산할 수 있습니다.

3.4 평가 증명

f(a) = v를 증명하려면:

아이디어: f(x) - v는 x = a에서 0이므로, (x - a)로 나누어 떨어짐

```
q(x) = (f(x) - v) / (x - a)
```

증명: π = q(τ)·G

검증 (페어링 사용):

```
e(C - v·G, H) = e(π, τH - a·H)
```

이 등식이 성립하면:

- e(f(τ)·G - v·G, H) = e(q(τ)·G, (τ-a)·H)
- f(τ) - v = q(τ)·(τ - a)
- 다항식 나머지 정리에 의해 f(a) = v

---

4. Groth16의 구조

4.1 R1CS에서 QAP로

3편에서 다룬 것처럼, 계산 문제는 다음으로 변환됩니다:

R1CS (Rank-1 Constraint System):

```
(A·s) ⊙ (B·s) = (C·s)
```

QAP (Quadratic Arithmetic Program):

```
A(x)·B(x) - C(x) = H(x)·Z(x)
```

여기서:

- A(x), B(x), C(x)는 witness 벡터 s로 결정되는 다항식
- Z(x) = (x-1)(x-2)...(x-n)은 소거 다항식
- H(x)는 몫 다항식

4.2 Groth16의 증명 구조

Groth16 증명 π는 **3개의 그룹 원소**로 구성됩니다[^5]:

```
π = (A, B, C)
```

여기서:

- **A** ∈ G_1
- **B** ∈ G_2
- **C** ∈ G_1

BLS12-381 기준:

- G_1 점: 48 bytes (압축 시)
- G_2 점: 96 bytes (압축 시)
- **총 증명 크기: 192 bytes**

[^5]: Groth, J. (2016). *On the Size of Pairing-Based Non-interactive Arguments*.

4.3 증명 생성 (Prover)

Prover는 witness $w = (w_1, w_2, ..., w_m)$를 알고 있습니다.

랜덤 블라인딩 값 r, s를 선택하고:

```
A = α + Σᵢ wᵢ·uᵢ(τ) + r·δ      (G₁에서)
B = β + Σᵢ wᵢ·vᵢ(τ) + s·δ      (G₂에서)
C = (Σᵢ wᵢ·(β·uᵢ(τ) + α·vᵢ(τ) + wᵢ(τ))/δ) + H(τ)·Z(τ)/δ + A·s + B·r - r·s·δ
```

여기서 α, β, δ는 Trusted Setup에서 생성된 비밀 값들입니다.

4.4 검증 (Verifier)

Verifier는 **단 하나의 페어링 등식**을 확인합니다:

```
e(A, B) = e(α, β) · e(Σᵢ xᵢ·[Lᵢ(τ)]₁, [γ]₂) · e(C, [δ]₂)
```

여기서:

- x_i는 공개 입력 (public inputs)
- $L_i(\tau)$는 공개 입력에 대응하는 다항식
- α, β, γ, δ는 검증 키의 일부

검증 비용:

- 3번의 페어링 연산
- 공개 입력 수에 비례하는 스칼라 곱
- **회로 크기와 무관!**

---

5. 왜 3개의 점으로 충분한가

5.1 Pinocchio와의 비교

Groth16 이전의 **Pinocchio** (PGHR13)는 **8개의 그룹 원소**가 필요했습니다[^6]:

- 4개의 G_1 점
- 4개의 G_2 점

Groth16은 이를 **3개로 줄였습니다**:

- 2개의 G_1 점 (A, C)
- 1개의 G_2 점 (B)

[^6]: Parno, B., et al. (2013). *Pinocchio: Nearly Practical Verifiable Computation*. S&P 2013.

5.2 핵심 아이디어

Groth16의 핵심 최적화:

1. **비대칭 페어링 활용**:
   - G_1과 G_2를 다르게 사용
   - A, C를 G_1에, B만 G_2에 배치
2. **α, β 시프트**:
   - A에 α를 더하고, B에 β를 더함
   - 검증 시 e(α, β) 항으로 일관성 강제
3. **δ로 나누기**:
   - private witness 관련 항을 δ로 나눔
   - Prover가 witness 없이 증명을 만들 수 없게 함
4. **랜덤화 (r, s)**:
   - 영지식성 보장
   - 증명을 랜덤하게 만들어 정보 누출 방지

5.3 직관적 이해

왜 이것이 작동하는가?

페어링 등식을 전개하면:

```
e(A, B) = e(α + a + rδ, β + b + sδ)
        = e(α, β) · e(α, b+sδ) · e(a+rδ, β) · e(a+rδ, b+sδ)
```

이 복잡한 표현이 올바른 witness를 사용했을 때만 검증 등식과 일치합니다.

Prover가 witness 없이 유효한 (A, B, C)를 만들려면:

- α, β, δ, γ의 이산로그를 알아야 함
- 이는 Trusted Setup이 안전하면 불가능

---

6. Trusted Setup의 구조

6.1 두 단계 Setup

Groth16의 Trusted Setup은 **두 단계**로 구성됩니다[^7]:

Phase 1: Powers of Tau (범용적)

- 모든 회로에 재사용 가능
- τ의 거듭제곱만 생성
- Perpetual Powers of Tau 세레모니로 공유

Phase 2: Circuit-Specific (회로별)

- 특정 회로의 R1CS/QAP에 의존
- α, β, γ, δ 및 회로별 파라미터 생성
- 회로마다 별도 세레모니 필요

[^7]: Bowe, S., Gabizon, A., & Miers, I. (2017). *Scalable Multi-party Computation for zk-SNARK Parameters*. https://eprint.iacr.org/2017/1050

6.2 Phase 1: Powers of Tau

생성되는 값들:

```
G₁: [τⁱ]₁ for i = 0, 1, ..., n
G₂: [τⁱ]₂ for i = 0, 1, ..., n (일부)
G₁: [α·τⁱ]₁, [β·τⁱ]₁ for i = 0, 1, ..., n
```

MPC 프로토콜:

1. 첫 참여자가 랜덤 τ_1을 선택, 거듭제곱 계산
2. 다음 참여자가 τ_2를 선택, 기존 값에 적용: $\tau = \tau_1 \cdot \tau_2$
3. 반복...
4. 최종 $\tau = \tau_1 \cdot \tau_2 \cdot \ldots \cdot \tau_n$

보안 가정: 참여자 중 **한 명이라도** 자신의 τ_i를 삭제하면 전체 τ는 복구 불가능

6.3 Phase 2: Circuit-Specific

회로의 QAP 다항식 $u_i(x)$, $v_i(x)$, $w_i(x)$를 사용하여:

```
Proving Key:
- [α]₁, [β]₁, [δ]₁
- [τⁱ·Z(τ)/δ]₁ for quotient polynomial
- [(β·uᵢ(τ) + α·vᵢ(τ) + wᵢ(τ))/δ]₁ for private inputs
- [(β·uᵢ(τ) + α·vᵢ(τ) + wᵢ(τ))/γ]₁ for public inputs

Verification Key:
- [α]₁, [β]₂, [γ]₂, [δ]₂
- [(β·uᵢ(τ) + α·vᵢ(τ) + wᵢ(τ))/γ]₁ for public inputs
```

6.4 Proving Key vs Verification Key

Key	크기	용도
Proving Key	O(회로 크기) - 수 GB 가능	Prover가 증명 생성에 사용
Verification Key	O(공개 입력 수) - 수 KB	Verifier가 검증에 사용

비대칭성:

- Prover: 무거운 계산, 큰 키 필요
- Verifier: 가벼운 계산, 작은 키 필요

이것이 블록체인에서 Groth16이 인기 있는 이유입니다. 온체인 검증 비용이 낮습니다.

---

7. 보안 분석

7.1 보안 모델

Groth16의 보안은 **Generic Group Model (GGM)**에서 증명되었습니다[^8].

가정:

- 그룹 연산 외에 그룹 구조를 이용할 방법이 없음
- 이산로그 문제가 어려움
- q-SDH (Strong Diffie-Hellman) 가정

[^8]: Fuchsbauer, G., Kiltz, E., & Loss, J. (2018). *The Algebraic Group Model and its Applications*. CRYPTO 2018.

7.2 Soundness (건전성)

Knowledge Soundness: 유효한 증명을 만들 수 있다면, witness를 추출할 수 있음

Groth16은 **knowledge soundness**를 만족합니다:

- Extractor가 Prover의 대수적 표현에서 witness를 추출
- 단, **simulation extractability는 부족** (증명 변조 가능)

7.3 Zero-Knowledge (영지식성)

Perfect Zero-Knowledge: 시뮬레이터가 witness 없이 구분 불가능한 증명 생성 가능

Groth16의 영지식성:

- 랜덤 r, s가 A, B, C를 랜덤화
- 시뮬레이터는 trapdoor (τ, α, β, γ, δ)를 사용하여 가짜 증명 생성
- 실제 증명과 통계적으로 구분 불가능

7.4 알려진 한계

Malleability (변조 가능성):

- Groth16 증명은 **re-randomizable** (재랜덤화 가능)
- 유효한 증명 (A, B, C)에서 다른 유효한 증명 (A', B', C') 생성 가능

- 일부 응용에서 문제 (예: 서명)

해결책:

- Simulation Extractable 변형 (Bowe-Gabizon 등)
- 추가 요소를 증명에 포함

---

8. 실제 사용 사례

8.1 Zcash Sapling

2018년 10월, Zcash는 **Sapling 업그레이드**로 Groth16으로 전환했습니다[^9]:

성능 개선 (Sprout 대비):

- 증명 시간: 40초 → 7초 (81% 감소)
- 메모리: 3GB → 40MB (97% 감소)
- 모바일 지갑 지원 가능

추가 최적화:

- BLS12-381 커브 (128비트 보안)
- Pedersen Hash (SHA-256 대체)
- Jubjub 내장 커브

[^9]: Electric Coin Company. (2018). *What's New in Sapling*.

8.2 Filecoin PoRep

Filecoin은 **저장 증명(Proof of Replication)**에 Groth16을 사용합니다:

- 32GB 섹터당 ~10분 증명 시간
- 증명 크기: 192 bytes
- 온체인 검증: ~10ms

Powers of Tau 세레모니:

- 100M 제약조건 지원 (BLS12-381 최대)
- 수백 명 참여

8.3 zkRollup

많은 zkRollup이 Groth16을 사용하거나 사용했습니다:

- **Tornado Cash**: 이더리움 프라이버시 믹서
- **Semaphore**: 익명 시그널링
- **ZK-Sync (초기)**: L2 스케일링

온체인 비용:

- Groth16 검증: ~200,000 gas
- PLONK 검증: ~300,000 gas

- STARK 검증: ~2,000,000 gas

---

9. Groth16 vs 다른 증명 시스템

9.1 비교표

특성	Groth16	PLONK	STARK
증명 크기	~200 bytes	~400 bytes	~40 KB
검증 시간	~1.2ms	~2ms	~10ms
증명 시간	중간	중간	빠름
Trusted Setup	회로별	범용	없음
양자 저항	❌	❌	☑
투명성	❌	부분적	☑

9.2 언제 Groth16을 선택하는가

Groth16이 적합한 경우:

- 증명 크기가 중요 (온체인 비용)
- 검증 속도가 중요
- 회로가 고정됨 (자주 바뀌지 않음)
- 양자 컴퓨터 위협이 당장 없음

다른 선택이 나은 경우:

- 회로가 자주 변경됨 → PLONK, Halo2
- Trusted Setup을 피하고 싶음 → STARK
- 양자 저항이 필요 → STARK
- 재귀 증명이 필요 → Halo2, Nova

---

10. 마치며: 우아함의 대가

Groth16은 **우아한 최적화**의 결과물입니다:

- 8개 원소 → 3개 원소
- 4번 페어링 → 3번 페어링
- 여전히 가장 작은 증명 크기

하지만 **대가**가 있습니다:

- **회로별 Trusted Setup**: 새 회로마다 새 세레모니
- **비범용성**: 회로가 고정되어야 함
- **양자 취약성**: 페어링 기반이므로 양자 컴퓨터에 취약

ZKP 세계는 계속 발전하고 있습니다:

- **PLONK** (2019): 범용 Setup
- **Halo/Halo2** (2019-2020): Trusted Setup 제거
- **Nova** (2022): 효율적인 재귀
- **STARK/FRI**: 투명성과 양자 저항

하지만 Groth16은 여전히 **"증명 크기가 중요한 곳"**에서 표준으로 남아있습니다.

다음 편에서는 **Filecoin의 PoRep**를 다룹니다:

- 저장 증명의 필요성
- Groth16의 대규모 응용
- SDR과 PoRep 회로

시리즈 로드맵

1편: 비밀을 증명한다는 것 ✓
2편: 컴퓨터가 이해하는 수학 ✓
3편: 프로그램을 다항식으로 - 산술화 ✓
4편: Zcash Sprout - 최초의 프라이버시 암호화폐 ✓
5편: Groth16의 마법 (현재) ✓
6편: Filecoin의 PoRep - 저장 증명
7편: PLONK - 범용 SNARK의 시대
...

Appendix A: Groth16 수학 요약

A.1 Setup

입력: R1CS (A, B, C 행렬), 보안 파라미터 λ

출력: (pk, vk)

1. 랜덤 선택: $\tau, \alpha, \beta, \gamma, \delta \leftarrow F^*$
2. QAP 다항식 계산: $u_i(x), v_i(x), w_i(x), Z(x)$
3. Proving Key 생성
4. Verification Key 생성
5. $\tau, \alpha, \beta, \gamma, \delta$ 삭제

A.2 Prove

입력: pk, witness $w = (w_1, \ldots, w_m)$

출력: $\pi = (A, B, C)$

1. 랜덤 선택: $r, s \leftarrow F$
2. $A = [\alpha + \Sigma_i w_i \cdot u_i(\tau) + r \cdot \delta]_1$
3. $B = [\beta + \Sigma_i w_i \cdot v_i(\tau) + s \cdot \delta]_2$
4. $C = [\Sigma_i(\text{private}) w_i \cdot (\beta \cdot u_i(\tau) + \alpha \cdot v_i(\tau) + w_i(\tau))/\delta + h(\tau) \cdot Z(\tau)/\delta + A \cdot s + r \cdot B - r \cdot s \cdot \delta]_1$

A.3 Verify

입력: vk, public inputs $x = (x_1, ..., x_l)$, $\pi = (A, B, C)$

출력: accept/reject

1. $L = \Sigma_i x_i \cdot [L_i]_1$ (공개 입력 선형 결합)
2. 검증: $e(A, B) = e([\alpha]_1, [\beta]_2) \cdot e(L, [\gamma]_2) \cdot e(C, [\delta]_2)$

References

1. Groth, J. (2016). *On the Size of Pairing-Based Non-interactive Arguments*. EUROCRYPT 2016. https://eprint.iacr.org/2016/260
2. Alin Tomescu. *Groth16*. https://alinush.github.io/groth16
3. Vitalik Buterin. (2017). *Exploring Elliptic Curve Pairings*. https://medium.com/@VitalikButerin/exploring-elliptic-curve-pairings-c73c1864e627
4. Kate, A., Zaverucha, G., & Goldberg, I. (2010). *Polynomial Commitments*. ASIACRYPT 2010. https://www.iacr.org/archive/asiacrypt2010/6477178/6477178.pdf
5. Parno, B., et al. (2013). *Pinocchio: Nearly Practical Verifiable Computation*. IEEE S&P 2013.
6. Bowe, S., Gabizon, A., & Miers, I. (2017). *Scalable Multi-party Computation for zk-SNARK Parameters*. https://eprint.iacr.org/2017/1050
7. Fuchsbauer, G., Kiltz, E., & Loss, J. (2018). *The Algebraic Group Model and its Applications*. CRYPTO 2018.
8. Electric Coin Company. (2018). *What's New in Sapling*. https://electriccoin.co/blog/whats-new-in-sapling/
9. RisenCrypto. *Groth16 zkSNARK*. https://risencrypto.github.io/Groth16/
10. Dankrad Feist. (2020). *KZG Polynomial Commitments*. https://dankradfeist.de/ethereum/2020/06/16/kate-polynomial-commitments.html
11. ZK Podcast. *The Power of Tau*. https://zeroknowledge.fm/the-power-of-tau-or-how-i-learned-to-stop-worrying-and-love-the-setup/
12. Panther Protocol. *Groth16 Documentation*. https://docs.pantherprotocol.io/docs/learn/cryptographic-primitives/zk-snarks/groth16